

Lecture 1: Mathematical Foundations for Lattice-Based Cryptography

Groups, Rings, Fields, Modular Arithmetic,
Vector Spaces, Basis, and Lattices

Dr. Faisal Aslam

Lecture Notes — Session 1 (approx. 2 hours)

Purpose of this lecture. Every technical term used later in the thesis proposal — “lattice,” “modulus,” “ring $\mathbb{Z}_q[x]/(x^n + 1)$ ” — rests on a small set of algebraic building blocks. This lecture builds those blocks, one at a time, from the ground up: *Group* $\rightarrow$ *Subgroup* $\rightarrow$ *Ring* $\rightarrow$ *Field* $\rightarrow$ *Modular Arithmetic* $\rightarrow$ *Vector Space* $\rightarrow$ *Basis* $\rightarrow$ *Lattice*. Nothing here requires prior abstract algebra; every definition is followed immediately by concrete, worked examples.

Contents

1	Why Start With Algebra?	1
2	Sets and Binary Operations (Quick Refresher)	1
3	Groups	2
3.1	Subgroups	2
3.2	Cosets and Quotient Groups: Where $\mathbb{Z}_n$ Really Comes From	3
4	Rings	4
4.1	Zero Divisors: A Structural Reason Some Rings Aren’t Fields	5
5	Fields	6
6	The Modulus: Arithmetic That Wraps Around	8
6.1	Finding Inverses Without Guessing: The Extended Euclidean Algorithm	9
7	Vector Spaces, Basis, and Dimension	11
7.1	Testing Linear Independence: Row Operations and Row Echelon Form	11
7.2	Inner Products, Norms, and Orthogonality	14
7.3	Gram–Schmidt Orthogonalization	16
8	Lattices: A Formal Definition	17
8.1	Discrete Sets	17
8.2	The Definition	18
8.3	The Shortest Vector and the Minimum Distance $\lambda_1(L)$	18
8.4	Recovering the Basis Picture	19
8.5	The Determinant (Volume) of a Lattice	21
8.6	Minkowski’s First Theorem: Volume Forces a Short Vector to Exist	23
8.6.1	A Fully Formal Proof, via Blichfeldt’s Lemma	25
9	Summary Table	30
10	Full List of Exercises (Assign as Homework Before Lecture 2)	31

1 Why Start With Algebra?

Lattice-based cryptography is, at its core, about doing arithmetic on *vectors* whose coordinates live in a *finite, wraparound number system*, arranged according to strict algebraic rules. Before we can even state what a lattice is, we need precise answers to questions like:

- What does it mean for a set of numbers to have a well-behaved notion of “add” and “multiply”? (**Groups, Rings, Fields**)
- What happens when arithmetic “wraps around,” like a clock? (**Modulus**)
- What does it mean to build every point in a space out of a small number of building-block directions? (**Vector spaces, Basis**)
- What happens when we only allow *whole-number* combinations of those building blocks, instead of any real-number combination? (**Lattice**)

This lecture answers each of these in order. By the end, the definition of a lattice (Section 8) should feel like the natural, inevitable next step, not an isolated new idea.

2 Sets and Binary Operations (Quick Refresher)

A **set** is just a collection of objects (numbers, vectors, symbols — anything). A **binary operation** on a set S is a rule that takes two elements of S and combines them into a third element, e.g., ordinary addition $+$ takes two numbers and produces their sum. We write $a * b$ for a general binary operation.

An operation is **closed** on S if combining two elements of S always gives another element of S (never something outside S). This sounds obvious but fails more often than you’d expect — see Example 1 below.

Example: Closure Can Fail

Let $S = \{1, 2, 3, 4, 5\}$ (just these five numbers) and let $*$ be ordinary addition. Then $2 * 4 = 6$, but $6 \notin S$. So ordinary addition is *not* closed on S . This is exactly the kind of problem modular arithmetic (Section 5) is built to fix.

3 Groups

Definition: Group

A set G together with a binary operation $*$ is called a **group**, written $(G, *)$, if all four of the following hold:

(G1) **Closure:** for all $a, b \in G$, $a * b \in G$.

(G2) **Associativity:** for all $a, b, c \in G$, $(a * b) * c = a * (b * c)$.

(G3) **Identity:** there exists an element $e \in G$ such that $a * e = e * a = a$ for all $a \in G$.

(G4) **Inverses:** for every $a \in G$, there exists $a^{-1} \in G$ such that $a * a^{-1} = a^{-1} * a = e$.

If, in addition, $a * b = b * a$ for all $a, b \in G$ (the order doesn’t matter), the group is called **abelian** (commutative).

Example: Groups You Already Know

- $(\mathbb{Z}, +)$ — the integers under addition. Identity is 0; the inverse of a is $-a$. This is an abelian group.
- $(\mathbb{R} \setminus \{0\}, \times)$ — nonzero real numbers under multiplication. Identity is 1; the inverse of a is $1/a$. Also abelian. (We had to remove 0 because 0 has no multiplicative inverse.)
- $(\mathbb{Z}, \times)$ — integers under multiplication — is **not** a group: 2 has no inverse in $\mathbb{Z}$ (we'd need $1/2$, which isn't an integer). This fails axiom (G4).

Remark: Why Cryptography Cares About Groups

Many classical cryptographic schemes (Diffie–Hellman, ElGamal, elliptic-curve cryptography) are built directly on top of group structure — the security relies on certain problems (like “given g and g^a , find a ”) being hard inside a specific group. Lattice-based cryptography instead builds on richer structures (rings and modules, coming next), but the group is always the first layer underneath.

Exercise: Try This Before Next Lecture

Is $(\mathbb{Z}_5 \setminus \{0\}, \times \bmod 5) = (\{1, 2, 3, 4\}, \times \bmod 5)$ a group? Check all four axioms directly by building the multiplication table. (You'll need Section 6 for what “mod 5” means if it's new to you — come back to this after reading that section.)

3.1 Subgroups

There is one more group-related idea we need before we can properly define a lattice: a group living *inside* another group.

Definition: Subgroup

Let $(G, *)$ be a group. A subset $H \subseteq G$ is a **subgroup** of G if H is itself a group under the same operation $*$. In practice, this is easiest to check with a simple test: H is a subgroup of G if and only if (1) H is nonempty (usually: it contains the identity e), (2) H is closed under $*$ (for $a, b \in H$, $a * b \in H$), and (3) H is closed under inverses (for $a \in H$, $a^{-1} \in H$). Associativity is automatic, since it already held for all of G .

Example: Subgroups You Can Picture

- For any integer n , the set $n\mathbb{Z} = \{\dots, -2n, -n, 0, n, 2n, \dots\}$ (all multiples of n) is a subgroup of $(\mathbb{Z}, +)$: it contains 0, adding two multiples of n gives another multiple of n , and $-(kn)$ is also a multiple of n . For example, $2\mathbb{Z}$ is “all even integers.”
- The natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$ are **not** a subgroup of $(\mathbb{Z}, +)$: $\mathbb{N}$ is closed under addition, but it fails closure under inverses — $1 \in \mathbb{N}$, but $-1 \notin \mathbb{N}$.

Looking Ahead: Why This Matters in Two Sections

Keep the example $n\mathbb{Z} \subseteq \mathbb{Z}$ in mind. In Section 8, we will define a lattice as a special kind of subgroup of $\mathbb{R}^n$ — and $n\mathbb{Z}$ is exactly a simple, one-dimensional lattice in disguise: an evenly-spaced, discrete set of points, closed under addition and negation, sitting inside the larger group $\mathbb{Z}$ (or $\mathbb{R}$).

3.2 Cosets and Quotient Groups: Where $\mathbb{Z}_n$ Really Comes From

Section 6 will introduce $\mathbb{Z}_n$ informally, as “remainders with wraparound arithmetic.” That’s the right intuition, but it’s worth seeing the fully rigorous construction now, because it is a direct, satisfying payoff of the subgroup idea — and it is the same construction (“take a group, collapse it by a subgroup”) that shows up again later when we build the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ that Kyber and Dilithium actually use.

Definition: Coset

Let H be a subgroup of an abelian group $(G, +)$. For $a \in G$, the **coset** $a+H$ is the set $\{a+h : h \in H\}$ — “shift H by a .” Two elements $a, b \in G$ turn out to produce the *same* coset, $a+H = b+H$, exactly when $a-b \in H$. This gives a clean way to sort all of G into non-overlapping “buckets,” each one a shifted copy of H .

Example: The Cosets of $3\mathbb{Z}$ in $\mathbb{Z}$

Take $G = \mathbb{Z}$ and $H = 3\mathbb{Z} = \{\dots, -3, 0, 3, 6, \dots\}$ (a subgroup, per the “Subgroups You Can Picture” example in Section 3.1). The cosets are:

- $0 + 3\mathbb{Z} = \{\dots, -3, 0, 3, 6, \dots\}$ (multiples of 3)
- $1 + 3\mathbb{Z} = \{\dots, -2, 1, 4, 7, \dots\}$ (remainder 1 when divided by 3)
- $2 + 3\mathbb{Z} = \{\dots, -1, 2, 5, 8, \dots\}$ (remainder 2 when divided by 3)

Every integer falls into exactly one of these three buckets — and which bucket it falls into is precisely its remainder mod 3. There is no fourth distinct coset: $3 + 3\mathbb{Z} = 0 + 3\mathbb{Z}$ (check: $3 - 0 = 3 \in 3\mathbb{Z}$), so the pattern repeats.

Definition: Quotient Group

The set of all distinct cosets of H in G , with addition defined by $(a+H) + (b+H) := (a+b)+H$, is itself a group — the **quotient group**, written G/H (“ $G \bmod H$ ”). Its identity element is the coset $0 + H = H$ itself.

Remark: $\mathbb{Z}_n$ Is Not Just Like $\mathbb{Z}/n\mathbb{Z}$ — It Is $\mathbb{Z}/n\mathbb{Z}$

Compare the coset example above to Section 6: the three cosets of $3\mathbb{Z}$ in $\mathbb{Z}$ correspond *exactly* to the three elements $\{0, 1, 2\}$ of $\mathbb{Z}_3$, and coset addition matches addition mod 3 exactly. This is not a coincidence or an analogy — it is a theorem: $\mathbb{Z}_n$, as defined informally in Section 6, is precisely the quotient group $\mathbb{Z}/n\mathbb{Z}$. “Wraparound arithmetic” and “sorting integers into cosets of $n\mathbb{Z}$ ” are two descriptions of the exact same object. This is the rigorous justification for a claim we made without proof in Section 6: that $\mathbb{Z}_n$ really is a well-defined group (in fact ring) under addition and multiplication with wraparound — it inherits this structure directly from being a quotient of $\mathbb{Z}$.

Exercise: Practice

Let $H = 4\mathbb{Z} \subseteq \mathbb{Z}$. List all distinct cosets of H in $\mathbb{Z}$, and confirm there are exactly 4 of them, matching $\mathbb{Z}_4 = \{0, 1, 2, 3\}$. Which coset does -5 belong to?

4 Rings

A ring is what you get when you ask a set to support *two* binary operations that play two structurally different roles — one behaves like addition (fully invertible), the other behaves like multiplication (weaker, no inverses required). We always *label* these two operations “+” and “×,” but — and this is worth being precise about, since it’s easy to misread — **the labels do not mean the operations have to literally be**

school-arithmetic addition and multiplication. They can be almost any two operations at all, as long as together they satisfy the axioms below. “+” and “×” are names for the *roles* the two operations play, not a requirement about what the operations must concretely do.

Definition: Ring

A set R with two binary operations $+$ and $\times$ is a **ring** $(R, +, \times)$ if:

(R1) $(R, +)$ is an abelian group (with identity element usually written 0).

(R2) $\times$ is associative: $(a \times b) \times c = a \times (b \times c)$.

(R3) The distributive laws hold: $a \times (b + c) = (a \times b) + (a \times c)$ and $(a + b) \times c = (a \times c) + (b \times c)$.

Note: a ring does *not* need multiplicative inverses, and multiplication need not be commutative in general (though in every ring we use in this thesis, it will be — these are called **commutative rings**). If there is an element $1 \in R$ with $a \times 1 = 1 \times a = a$ for all a , we say R is a **ring with unity**.

Remark: “+” and “×” Are Role Labels, Not a Restriction to Arithmetic

Compare this to groups (Section 3): a group has *one* operation, and we wrote it as a generic $*$ precisely to emphasize that it could be anything (addition, multiplication, matrix multiplication, function composition — whatever satisfies the axioms). A ring needs *two* operations, and it turns out we always call them “+” and “×” — but this is a naming convention for the two *roles*, not a claim that the operations must be ordinary numeric addition and multiplication. Concretely:

- **$n \times n$ matrices:** “+” is ordinary matrix addition (this forms an abelian group), and “×” is matrix *multiplication* (associative, distributes over +) — but matrix multiplication is **not** commutative in general, which is fine, since a ring’s axioms above don’t require it (only *commutative* rings require it, and this one wouldn’t qualify as one).
- **Boolean rings** (used in digital logic): “+” is XOR and “×” is AND on $\{0, 1\}$ — nothing to do with school arithmetic at all, yet every ring axiom above holds.

So whenever you meet a new ring in this course, the first question to ask is not “is + really addition and is × really multiplication?” but rather: *what are the two operations here, and do they satisfy axioms (R1)–(R3)?* The labels “+”/“×” just tell you which operation is playing which role once you’ve checked that.

Example: Rings You Already Know

- $(\mathbb{Z}, +, \times)$ — the integers form a commutative ring with unity 1. Notice 2 still has no multiplicative inverse, and that’s fine — rings don’t require one.
- **Polynomial rings**, e.g. $\mathbb{Z}[x]$: the set of all polynomials with integer coefficients (like $3x^2 - x + 7$), with ordinary polynomial addition and multiplication. This is a ring for exactly the same reason $\mathbb{Z}$ is.

Looking Ahead: The Ring That Kyber and Dilithium Actually Use

Both Kyber and Dilithium do their arithmetic inside a specific ring of the form

$$R_q = \mathbb{Z}_q[x]/(x^n + 1),$$

read as: “polynomials with coefficients taken modulo q (Section 6), where we also reduce powers of x using the rule $x^n \equiv -1$.” This looks intimidating right now, but it is built from exactly the pieces in this lecture: a polynomial ring (this section), with coefficients from $\mathbb{Z}_q$ (Section 6), further “folded” by a fixed rule. We will unpack this fully in a later lecture — for now, just notice that “ring” is not an abstract curiosity; it’s the literal object the cryptography is built on.

4.1 Zero Divisors: A Structural Reason Some Rings Aren't Fields

We will soon see (Section 5) that $\mathbb{Z}_n$ is a field exactly when n is prime. Section 6.1 will justify this computationally, via gcd. Here is the deeper, structural reason *why* — worth seeing now, while we're still inside general ring theory.

Definition: Zero Divisor

In a ring R , a nonzero element a is a **zero divisor** if there exists some *nonzero* $b \in R$ with $a \times b = 0$. A commutative ring with unity that has *no* zero divisors (other than needing a or b to be 0) is called an **integral domain**.

Example: Zero Divisors in $\mathbb{Z}_6$, Absent in $\mathbb{Z}_5$

In $\mathbb{Z}_6$: $2 \times 3 = 6 \equiv 0 \pmod{6}$, but neither 2 nor 3 is 0. So 2 and 3 are both zero divisors — $\mathbb{Z}_6$ is *not* an integral domain. Contrast this with $\mathbb{Z}_5$: checking every pair of nonzero elements (1, 2, 3, 4), no product is ever $\equiv 0 \pmod{5}$ (e.g., $2 \times 3 = 6 \equiv 1$, $4 \times 4 = 16 \equiv 1$, etc.) — $\mathbb{Z}_5$ has no zero divisors.

Remark: Why This Explains the Prime/Composite Split

Fact: every field is automatically an integral domain (if $a \neq 0$ and $ab = 0$, multiply both sides by a^{-1} to get $b = 0$ — so no nonzero b can pair with a nonzero a to give 0). Contrapositive: *if a ring has a zero divisor, it cannot be a field*. Now, $\mathbb{Z}_n$ has a zero divisor exactly when n is composite: write $n = jk$ with $1 < j, k < n$; then $j \times k = n \equiv 0 \pmod{n}$, with neither j nor k equal to 0 in $\mathbb{Z}_n$. So every composite modulus produces zero divisors, which immediately rules out being a field — a purely structural explanation, with no gcd computation needed, of the same fact Section 6.1 reaches computationally.

Exercise: Practice

Find a pair of zero divisors in $\mathbb{Z}_{12}$ (there are several valid answers). Then explain, using the Fact above, why $\mathbb{Z}_{12}$ cannot be a field — without computing any inverses directly.

5 Fields

Definition: Field

A **field** is a set F with two binary operations $+$ and $\times$ such that:

(F1) $(F, +)$ is an abelian group, over *all* of F (identity element 0).

(F2) $(F \setminus \{0\}, \times)$ is an abelian group, over F **with 0 removed** (identity element 1).

(F3) The distributive law holds: $a \times (b + c) = (a \times b) + (a \times c)$.

Informally: a field is a set where you can add, subtract, multiply, and divide (by anything except 0) and always stay inside the set. In short — **a field is two abelian groups sharing one set, glued together by distributivity**.

Remark: Why 0 Has to Be Excluded, and It's Not Optional

In *any* structure with a distributive law, $0 \times x = 0$ for every x (this follows from distributivity itself, not from an extra rule: $0 \times x = (0 + 0) \times x = 0 \times x + 0 \times x$, so subtracting $0 \times x$ from both sides gives $0 = 0 \times x$). So 0 could only have a multiplicative inverse if $0 \times x = 1$ for some x — but the left side is always 0, never 1 (as long as $1 \neq 0$, which we always assume). So 0 is permanently excluded from having an inverse, in every field, without exception; it isn't a special case someone chose, it's forced by the arithmetic.

Definition: Subtraction and Division Are *Derived*, Not Extra

Neither subtraction nor division is a new, independent operation requiring its own axiom — both are just notation for “combine with an inverse,” one level for each operation:

$$a - b := a + (-b), \quad \frac{a}{b} := a \times b^{-1} \quad (b \neq 0),$$

where $-b$ is guaranteed to exist by $(F, +)$ being a group (F1), and b^{-1} is guaranteed to exist, for every *nonzero* b , by $(F \setminus \{0\}, \times)$ being a group (F2). So “a field supports division” is not an extra requirement on top of the definition — it falls straight out of (F2) once every nonzero element is already promised an inverse. This is exactly why (F2) excludes 0: since 0 can never have an inverse (see the remark above), “dividing by 0” can never be defined this way, no matter what field you're in.

Example: Division in $\mathbb{Z}_5$, Worked Out

What is $4 \div 3$ in $\mathbb{Z}_5$? First find $3^{-1} \bmod 5$: $3 \times 2 = 6 \equiv 1 \pmod{5}$, so $3^{-1} = 2$. Then $4 \div 3 := 4 \times 3^{-1} = 4 \times 2 = 8 \equiv 3 \pmod{5}$. Check: $3 \times 3 = 9 \equiv 4 \pmod{5}$ — correct, exactly as “ $4 \div 3 = 3$ ” should verify.

Remark: The One-Line Takeaway

Subtraction is the additive inverse; division is the multiplicative inverse. Same trick, applied once per operation — $a - b$ means “add a and b 's additive inverse,” a/b means “multiply a by b 's multiplicative inverse.” The one asymmetry between them traces directly back to (F1) vs. (F2): *every* element of F has an additive inverse, since $(F, +)$ is a group over *all* of F — so subtraction always works. But *only nonzero* elements have a multiplicative inverse, since $(F \setminus \{0\}, \times)$ is a group only after removing 0 — so division by b only ever works when $b \neq 0$.

Fact: Every Field Is a Ring

Every field F (Definition above) is automatically a commutative ring with unity — in fact, an **integral domain** (Section 4.1). *Why*: (F1) is exactly ring axiom (R1). Associativity and distributivity of $\times$ (ring axioms (R2)–(R3)) are inherited directly from $\times$ being a group operation on $F \setminus \{0\}$ together with (F3). So a field satisfies every ring axiom, plus two extra guarantees stacked on top: $\times$ is commutative, and every *nonzero* element is invertible. This is why some textbooks instead *define* a field as “a commutative ring with unity in which every nonzero element has a multiplicative inverse” — that phrasing and the definition above pick out exactly the same objects; it's just built in the reverse order (starting from “ring” and adding a condition, instead of starting from two groups and gluing them together). Finally, a field is automatically an integral domain too: if $ab = 0$ with $a \neq 0$, multiplying both sides by a^{-1} forces $b = a^{-1}(ab) = a^{-1} \cdot 0 = 0$, so there can be no zero divisors.

Example: Fields and Non-Fields

- $\mathbb{Q}$ (rationals), $\mathbb{R}$ (reals), $\mathbb{C}$ (complex numbers) are all fields — you can divide by any nonzero element and stay inside the set.
- $\mathbb{Z}$ is **not** a field: $2 \in \mathbb{Z}$ has no multiplicative inverse in $\mathbb{Z}$ (again, $1/2 \notin \mathbb{Z}$). $\mathbb{Z}$ is a ring, but not a field.
- $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$ under addition and multiplication mod 5 *is* a field — every nonzero element has an inverse (you checked this, or will check this, in the Section 3 subgroup/group exercise). Fields built this way, from $\mathbb{Z}_p$ with p prime, are called **finite fields** or **Galois fields**, written $\mathbb{F}_p$ or $GF(p)$.

Remark: The One-Sentence Summary So Far

Group $\subset$ Ring $\subset$ Field, in the sense that a field is a ring with extra structure (division), and a ring is built from a group with extra structure (a second operation). Each step adds a capability: groups let you undo one operation (inverses); rings add a second, distributive operation; fields guarantee *both* operations are fully invertible (except division by 0).

6 The Modulus: Arithmetic That Wraps Around

Definition: Congruence Modulo n

For a fixed positive integer n (the **modulus**), two integers a and b are **congruent modulo n** , written

$$a \equiv b \pmod{n},$$

if n divides $(a - b)$ evenly — equivalently, a and b leave the same remainder when divided by n . The set of possible remainders, $\{0, 1, 2, \dots, n - 1\}$, together with addition and multiplication performed “with wraparound,” is written $\mathbb{Z}_n$ (or $\mathbb{Z}/n\mathbb{Z}$).

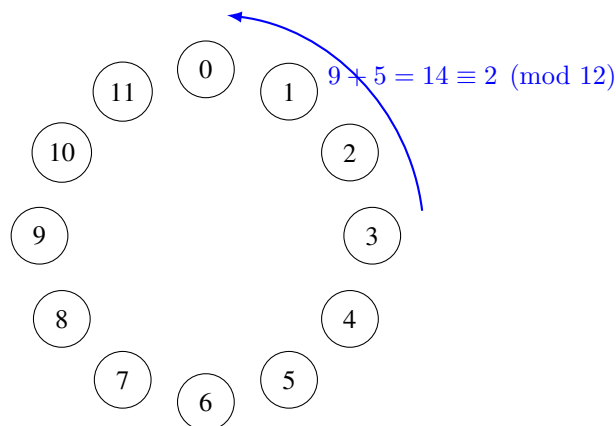


Figure 1: The clock analogy for modular arithmetic: $\mathbb{Z}_{12}$ works exactly like a 12-hour clock face. 9 o'clock plus 5 hours wraps around to 2 o'clock, i.e., $9 + 5 = 14 \equiv 2 \pmod{12}$.

Example: Computing Modulo

$17 \bmod 5$: divide 17 by 5 to get 3 remainder 2, so $17 \equiv 2 \pmod{5}$. Similarly, $23 \equiv 3 \pmod{10}$, and $-1 \equiv n - 1 \pmod{n}$ (e.g., $-1 \equiv 4 \pmod{5}$) — negative numbers wrap around too.

Remark: When Is $\mathbb{Z}_n$ a Ring? When Is It a Field?

$\mathbb{Z}_n$ is *always* a commutative ring with unity, for any $n \geq 2$. But it is a *field* only when $n = p$ is **prime**. If n is composite (e.g., $n = 6$), some nonzero elements fail to have multiplicative inverses — e.g., in $\mathbb{Z}_6$, there is no x with $2x \equiv 1 \pmod{6}$, since 2 shares a common factor with 6. This is exactly why cryptographic schemes that need a field (rather than just a ring) are built with a prime modulus q . (Section 4.1 gave the deeper structural reason: composite n always produces zero divisors, and a ring with zero divisors can never be a field.)

6.1 Finding Inverses Without Guessing: The Extended Euclidean Algorithm

So far, the only way we’ve found a multiplicative inverse mod n is trial and error (Exercise: “find x with $3x \equiv 1 \pmod{7}$ ” — you could just try $x = 1, 2, 3, \dots$ until one works). That’s fine for small numbers, but real cryptographic moduli are enormous (Kyber’s modulus is $q = 3329$; RSA moduli have hundreds of digits) — trial and error is completely impractical. We need an actual algorithm.

Definition: Greatest Common Divisor (GCD)

For integers a, b (not both zero), $\gcd(a, b)$ is the largest positive integer that divides both a and b . Two integers are **coprime** (or relatively prime) if $\gcd(a, b) = 1$.

Remark: Fact: When Does an Inverse Exist?

An element $a \in \mathbb{Z}_n$ has a multiplicative inverse if and only if $\gcd(a, n) = 1$. This is why every nonzero element of $\mathbb{Z}_p$ (for prime p) has an inverse: any a with $1 \leq a \leq p - 1$ automatically satisfies $\gcd(a, p) = 1$, since p ’s only divisors are 1 and p itself.

The classical **Euclidean Algorithm** computes $\gcd(a, b)$ by repeated division with remainder: $\gcd(a, b) = \gcd(b, a \bmod b)$, stopping when the remainder hits 0. The **Extended Euclidean Algorithm** does this same process but also tracks, at *every single step*, how the current remainder can be written as a combination $a \cdot x + b \cdot y$ (integers x, y) — and when the algorithm finishes with $\gcd(a, b) = 1$, the row where the remainder equals 1 hands you the inverse directly. The cleanest way to track this — much easier to follow than plain back-substitution — is a running **table** with one row per step.

Definition: The Extended Euclidean Algorithm, as a Table

To find $\gcd(a, b)$ together with integers x, y satisfying $a \cdot x + b \cdot y = \gcd(a, b)$, build a table with columns q (quotient), r (remainder), x, y , starting with two fixed initial rows and then one new row per division step:

step	q	r	x	y
−1	—	a	1	0
0	—	b	0	1
1	$q_1 = \lfloor r_{-1}/r_0 \rfloor$	$r_1 = r_{-1} - q_1 r_0$	$x_1 = x_{-1} - q_1 x_0$	$y_1 = y_{-1} - q_1 y_0$
2	$q_2 = \lfloor r_0/r_1 \rfloor$	$r_2 = r_0 - q_2 r_1$	$x_2 = x_0 - q_2 x_1$	$y_2 = y_0 - q_2 y_1$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Every single row satisfies the **invariant** $r_i = a \cdot x_i + b \cdot y_i$ by construction — this is the whole mechanism, and it’s worth checking explicitly on at least one row the first time you use this table, to convince yourself it isn’t magic. Stop once a row produces $r_i = 0$; the *previous* row’s r is $\gcd(a, b)$, and that row’s x, y are exactly the combination you want.

Example: Table Method, Short Version: $\gcd(7, 3)$ and $3^{-1} \bmod 7$

step	q	r	x	y	check: $7x + 3y$
-1	—	7	1	0	$7(1) + 3(0) = 7 \checkmark$
0	—	3	0	1	$7(0) + 3(1) = 3 \checkmark$
1	$\lfloor 7/3 \rfloor = 2$	$7 - 2(3) = 1$	$1 - 2(0) = 1$	$0 - 2(1) = -2$	$7(1) + 3(-2) = 1 \checkmark$
2	$\lfloor 3/1 \rfloor = 3$	$3 - 3(1) = 0$	$0 - 3(1) = -3$	$1 - 3(-2) = 7$	(stop: $r = 0$)

The row *before* r hit 0 is step 1: $r_1 = 1 = \gcd(7, 3)$, with $x_1 = 1, y_1 = -2$. So $7(1) + 3(-2) = 1$, i.e., $3 \times (-2) \equiv 1 \pmod{7}$, giving $3^{-1} \equiv -2 \equiv 5 \pmod{7}$ — matching what trial and error would also find, but now via a method with a clear, mechanical stopping point.

Example: Table Method, Longer Version: $15^{-1} \bmod 26$

Here the table earns its keep — more steps than the tiny example above, but the process is identical, row by row.

step	q	r	x	y
-1	—	26	1	0
0	—	15	0	1
1	$\lfloor 26/15 \rfloor = 1$	$26 - 1(15) = 11$	$1 - 1(0) = 1$	$0 - 1(1) = -1$
2	$\lfloor 15/11 \rfloor = 1$	$15 - 1(11) = 4$	$0 - 1(1) = -1$	$1 - 1(-1) = 2$
3	$\lfloor 11/4 \rfloor = 2$	$11 - 2(4) = 3$	$1 - 2(-1) = 3$	$-1 - 2(2) = -5$
4	$\lfloor 4/3 \rfloor = 1$	$4 - 1(3) = 1$	$-1 - 1(3) = -4$	$2 - 1(-5) = 7$
5	$\lfloor 3/1 \rfloor = 3$	$3 - 3(1) = 0$	$3 - 3(-4) = 15$	$-5 - 3(7) = -26$

The row before r hit 0 is step 4: $r_4 = 1 = \gcd(26, 15)$, with $x_4 = -4, y_4 = 7$. **Check the invariant directly:** $26(-4) + 15(7) = -104 + 105 = 1 \checkmark$. Reading this modulo 26: $15 \times 7 \equiv 1 \pmod{26}$, so $15^{-1} \equiv 7 \pmod{26}$. **Final check:** $15 \times 7 = 105 = 4 \times 26 + 1 \equiv 1 \pmod{26}$. Correct.

Exercise: Practice

Use the table method above (not back-substitution, and not trial and error) to find $5^{-1} \bmod 11$. Build the full table — steps $-1, 0, 1, 2, \dots$ — and stop at the correct row. Then check your answer by direct multiplication.

Looking Ahead: Why Cryptography Loves the Modulus

Modular arithmetic gives us two things cryptography needs badly: (1) a **finite** number system (so keys and computations have a fixed, bounded size, no matter how much you compute), and (2) built-in **one-wayness** in places — e.g., given only $a \bmod n$, you cannot tell which of infinitely many integers a actually was. The Learning-With-Errors (LWE) problem from the proposal document hides its secret inside equations that live in exactly this kind of finite, modular number system (there, the modulus is usually called q).

Exercise: Practice

(a) Compute $23 \times 15 \bmod 7$. (b) List all elements of $\mathbb{Z}_8$ that *do not* have a multiplicative inverse, and explain why (hint: compare each element to 8). (c) Confirm $\mathbb{Z}_7$ is a field by checking that 3 has a multiplicative inverse mod 7 (find x such that $3x \equiv 1 \pmod{7}$).

7 Vector Spaces, Basis, and Dimension

Definition: Vector Space

A **vector space** over a field F is a set V (whose elements are called **vectors**) equipped with an addition operation $V \times V \rightarrow V$ and a scalar multiplication operation $F \times V \rightarrow V$, satisfying, for all $u, v, w \in V$ and all $a, b \in F$:

- (V1) **Closure of addition:** $u + v \in V$.
- (V2) **Associativity of addition:** $(u + v) + w = u + (v + w)$.
- (V3) **Commutativity of addition:** $u + v = v + u$.
- (V4) **Additive identity:** there exists $\vec{0} \in V$ with $v + \vec{0} = v$ for all v .
- (V5) **Additive inverses:** for every $v \in V$, there exists $-v \in V$ with $v + (-v) = \vec{0}$.
- (V6) **Compatibility of scalar multiplication with field multiplication:** $a(bv) = (ab)v$.
- (V7) **Identity element of scalar multiplication:** $1v = v$, where 1 is the multiplicative identity of F .
- (V8) **Distributivity, two directions:** $a(u + v) = au + av$ and $(a + b)v = av + bv$.

Axioms (V1)–(V5) say exactly “ $(V, +)$ is an abelian group” (Section 3) — nothing new there. Axioms (V6)–(V8) are the genuinely new content: they say scalar multiplication by field elements behaves consistently with the field’s own addition and multiplication (Sections 4–5). The most important example for us: $\mathbb{R}^n$, the set of all n -tuples of real numbers, is a vector space over $\mathbb{R}$ — and it’s a good exercise to mentally check each of (V1)–(V8) holds there before moving on.

Definition: Linear Combination, Span, Linear Independence

Given vectors $v_1, \dots, v_k \in V$ and scalars $c_1, \dots, c_k \in F$, the vector $c_1v_1 + c_2v_2 + \dots + c_kv_k$ is called a **linear combination** of $v_1, \dots, v_k$. The set of *all* linear combinations of a set of vectors is called their **span**. The vectors $v_1, \dots, v_k$ are **linearly independent** if the only way to write $0 = c_1v_1 + \dots + c_kv_k$ is with every $c_i = 0$ — informally, no vector in the set is “redundant” (expressible using the others).

7.1 Testing Linear Independence: Row Operations and Row Echelon Form

The definition above is a yes/no test stated abstractly in terms of the c_i ’s. For concrete vectors in $\mathbb{R}^n$, it is also a completely mechanical computation — and the standard tool for carrying it out, **Gaussian elimination**, is worth setting up properly here, since we will lean on it again once bases and lattices enter the picture.

Definition: Elementary Row Operations

Given a matrix, the following three operations on its rows are called **elementary row operations**:

- (E1) **Row swap:** interchange two rows, $R_i \leftrightarrow R_j$.
- (E2) **Row scaling:** multiply a row by a nonzero scalar, $R_i \rightarrow cR_i$ for some $c \neq 0$.
- (E3) **Row addition (replacement):** add a scalar multiple of one row to another, $R_i \rightarrow R_i + cR_j$ (for $i \neq j$).

Each operation is **reversible** by another operation of the same type: a swap undoes itself; scaling by c is undone by scaling by $1/c$; adding cR_j is undone by adding $-cR_j$. This reversibility is exactly why row operations never destroy information — every row of the new matrix is some combination of the old rows, and (because the operation can be undone) every old row is likewise recoverable as a combination of the new ones.

Fact: Row Operations Preserve the Span — and Hence Independence — of the Rows

Let $v_1, \dots, v_k$ be the rows of a matrix A . Applying any sequence of elementary row operations produces a new matrix A' whose rows $v'_1, \dots, v'_k$ **span exactly the same space** as $v_1, \dots, v_k$ did — because, by the remark above, each new row is some linear combination of the old rows, and each old row is recoverable as a combination of the new ones. In particular: $v_1, \dots, v_k$ are linearly independent **if and only if** $v'_1, \dots, v'_k$ are. Row operations are therefore “free” for this purpose — we can simplify a set of vectors as aggressively as we like without ever changing the yes/no answer to the independence question, exactly the same way row operations leave gcd, solution sets, and (as we’ll see in Section 8.5) a lattice’s determinant unchanged under the right kind of transformation.

Definition: Row Echelon Form

A matrix is in **row echelon form (REF)** if:

- (1) Every all-zero row (if any) lies below every nonzero row.
- (2) In each nonzero row, its first nonzero entry — the **leading entry** or **pivot** — lies strictly to the right of the leading entry of the row above it (so the pivots form a descending “staircase”).

Any matrix can be brought to row echelon form using finitely many elementary row operations — systematically eliminating entries below each pivot, top to bottom, is exactly **Gaussian elimination**. The number of nonzero rows in the resulting echelon form is called the **rank** of the matrix; by the Fact above, this number doesn’t depend on which particular sequence of row operations you used to get there.

Remark: Turning This Into an Independence Test

Stack $v_1, \dots, v_k \in \mathbb{R}^n$ as the k rows of a $k \times n$ matrix A , and row-reduce A to echelon form A' . Since row operations never change independence, $v_1, \dots, v_k$ are linearly independent **exactly when A' has no zero row** — equivalently, when $\text{rank}(A) = k$ (every row produced its own pivot). If instead a zero row appears, the original vectors are dependent — and the specific combination of row operations that produced that zero row *is* a witness: it records exactly which combination of the original rows sums to $\vec{0}$. The worked example below shows a clean way to read that combination off directly.

Example: Row Reduction in Action: Testing $v_1 = (1, 2, 3)$, $v_2 = (2, 3, 4)$, $v_3 = (1, 1, 1)$

Stack the vectors as rows, and — to make the eventual dependency relation easy to read off — augment the matrix on the right with the 3×3 identity, one column per original vector:

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 2 & 3 & 4 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 \end{array} \right] \begin{array}{l} \leftarrow v_1 \\ \leftarrow v_2 \\ \leftarrow v_3 \end{array}$$

The idea: whatever combination of rows we apply to the left block, we apply *identically* to the right block — so the right block always records “this row currently equals (this combination) of v_1, v_2, v_3 .”

Step 1: eliminate below the first pivot. Use R_1 (pivot in column 1) to clear column 1 in R_2 and R_3 :

$$R_2 \rightarrow R_2 - 2R_1, \quad R_3 \rightarrow R_3 - R_1.$$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & -1 & -2 & -2 & 1 & 0 \\ 0 & -1 & -2 & -1 & 0 & 1 \end{array} \right]$$

Step 2: eliminate below the second pivot. R_2 's leading entry is now in column 2; use it to clear column 2 in R_3 :

$$R_3 \rightarrow R_3 - R_2.$$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & -1 & -2 & -2 & 1 & 0 \\ 0 & 0 & 0 & 1 & -1 & 1 \end{array} \right]$$

This is row echelon form: two pivots (row 1 in column 1, row 2 in column 2), and one zero row — so $\text{rank} = 2 < 3 = k$, meaning v_1, v_2, v_3 **are linearly dependent**.

Reading off the dependency. Row 3's right-hand block, $(1, -1, 1)$, records exactly which combination of the *original* rows produced that zero row: $1 \cdot v_1 + (-1) \cdot v_2 + 1 \cdot v_3$. Check directly: $v_1 - v_2 + v_3 = (1, 2, 3) - (2, 3, 4) + (1, 1, 1) = (0, 0, 0) \checkmark$. So $c_1 = 1$, $c_2 = -1$, $c_3 = 1$ is a non-trivial solution — equivalently, $v_2 = v_1 + v_3$, which you can also confirm directly.

Remark: Why More Vectors Than Dimensions Forces Dependence

If $k > n$ (more vectors than the ambient dimension), the vectors are *automatically* dependent — no computation needed. Setting up $c_1 v_1 + \cdots + c_k v_k = \vec{0}$ gives n equations (one per coordinate) in $k > n$ unknowns $c_1, \dots, c_k$; row-reducing the coefficient matrix can produce at most n pivots (one per row), so at least $k - n$ of the unknowns are left as **free variables** — and a homogeneous system with a free variable always has a nontrivial solution (just set the free variable to 1 and solve for the rest). We will meet this exact fact again, in geometric disguise, once we reach lattices in Section 8.

Definition: Basis and Dimension

A set of vectors $B = \{b_1, \dots, b_k\}$ is a **basis** of a vector space V if (1) B is linearly independent, and (2) B **spans** V (every vector in V can be written as a linear combination of B). Every basis of a given vector space has the same number of elements k , called the **dimension** of V .

Example: Basis of $\mathbb{R}^2$

The vectors $e_1 = (1, 0)$ and $e_2 = (0, 1)$ form the “standard” basis of $\mathbb{R}^2$: any vector (a, b) is exactly $a \cdot e_1 + b \cdot e_2$. But this is far from the *only* basis — $\{(1, 1), (1, -1)\}$ is also a valid basis of $\mathbb{R}^2$ (check: they’re linearly independent, and any (a, b) can be written as a combination of them). A vector space almost always has infinitely many possible bases; some are more convenient to work with than others. **Keep this fact in mind — it becomes very important in Section 8.**

Remark: A Quick Shortcut When $k = n$

When there are exactly as many vectors as dimensions, there’s a fast special-case test that avoids row reduction entirely: stack $v_1, \dots, v_n$ as the columns (or rows) of a square matrix A and compute $\det(A)$. Nonzero determinant $\Rightarrow$ independent; zero determinant $\Rightarrow$ dependent. This is a quick yes/no check, but — unlike the row-reduction method above — it doesn’t by itself hand you the explicit nontrivial combination when the answer is “dependent”; for that, fall back on row reduction (optionally with the identity-augmentation trick above).

Exercise: Practice: Testing Linear Independence

For each set of vectors in $\mathbb{R}^3$ below, determine whether the set is linearly independent. If a set turns out to be **dependent**, don’t just say so — exhibit an explicit **non-trivial** solution (scalars c_1, c_2, c_3 , not all zero) to $c_1v_1 + c_2v_2 + c_3v_3 = \vec{0}$, and check it by substitution. Use row reduction (with the identity-augmentation trick from the worked example above) at least once, even where the answer is also visible by inspection.

(a) $v_1 = (2, 4, 6)$, $v_2 = (1, 2, 3)$, $v_3 = (0, 1, 0)$.

(b) $v_1 = (1, 0, 0)$, $v_2 = (1, 1, 0)$, $v_3 = (1, 1, 1)$.

(c) $v_1 = (1, 2, 1)$, $v_2 = (2, 1, 3)$, $v_3 = (4, 5, 5)$.

(Hint for part (c): unlike part (a), the dependency here isn’t visible by inspection — row-reduce the augmented matrix $[v_1; v_2; v_3 \mid I_3]$ exactly as in the worked example above.)

7.2 Inner Products, Norms, and Orthogonality

To make “good basis” (short, nearly-perpendicular vectors) versus “bad basis” (long, nearly-parallel vectors) precise rather than just intuitive, we need a way to measure length and angle.

Definition: Inner Product and Norm

For $x = (x_1, \dots, x_n), y = (y_1, \dots, y_n) \in \mathbb{R}^n$, the (standard) **inner product** is $\langle x, y \rangle = \sum_{i=1}^n x_i y_i$. The **norm** (length) of x is $\|x\| = \sqrt{\langle x, x \rangle}$. Two vectors are **orthogonal** if $\langle x, y \rangle = 0$ (the angle between them is 90°). A basis $\{b_1, \dots, b_k\}$ is an **orthogonal basis** if every pair b_i, b_j ($i \neq j$) is orthogonal.

Example: Computing Norms and Checking Orthogonality

For $x = (3, 4)$: $\|x\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$. For $x = (1, 1)$ and $y = (1, -1)$: $\langle x, y \rangle = (1)(1) + (1)(-1) = 0$, so these two vectors (the alternate basis of $\mathbb{R}^2$ from the previous example) are orthogonal — a nicer basis, geometrically, than it might have looked at first glance.

Remark: Extending the Inner Product to Complex Vectors

The formula $\langle x, y \rangle = \sum_i x_i y_i$ above is only correct for *real* vectors, $x, y \in \mathbb{R}^n$. It breaks down over $\mathbb{C}^n$: take a single complex coordinate, $x = (i)$. The formula gives $\langle x, x \rangle = i \cdot i = i^2 = -1$, so $\|x\| = \sqrt{-1}$ — not a real number at all, let alone a sensible “length.” A norm must be a non-negative real number, so this naive formula simply stops being a norm once complex entries are allowed.

The standard fix is to place a **complex conjugate** on one argument,

$$\langle x, y \rangle = \sum_{i=1}^n x_i \overline{y_i} \quad (\text{the Hermitian inner product}),$$

where $\overline{a + bi} = a - bi$. Now $\langle x, x \rangle = \sum_i x_i \overline{x_i} = \sum_i |x_i|^2$ — a sum of non-negative real numbers, always real and always ≥ 0 , exactly what a norm needs. And whenever every entry happens to be real, $\overline{y_i} = y_i$, so this formula collapses back *exactly* to the plain dot product in the defbox above — the conjugate only ever does work when the entries are genuinely complex, and is invisible (a no-op) on real vectors. This is why the definition above was stated over $\mathbb{R}^n$ specifically: every lattice in Lectures 1–4 lives in $\mathbb{R}^n$ or $\mathbb{Z}^n$, so the plain real formula is all we need for now — the moment complex vectors enter the picture, however, conjugating becomes required, not optional.

Looking Ahead: Where Complex Vectors Actually Show Up (Preview of Falcon, Lecture 4)

This isn’t a hypothetical fix-it-just-in-case. Falcon’s signing step (“fast Fourier sampling,” Lecture 4) embeds a ring element of $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ into $\mathbb{C}^n$ using the n complex roots of $x^n + 1$ — the **canonical embedding** — because in that embedding, the otherwise-awkward polynomial multiplication in R_q becomes simple coordinate-wise multiplication of complex numbers, which is exactly what makes fast Fourier sampling fast. Every norm and inner product computed inside that embedding *is* the Hermitian version defined above, not the plain real dot product. Keep this box in mind when Falcon’s sampler resurfaces in Session 5: part of why its floating-point implementation is so delicate to secure is that it’s performing real, finite-precision arithmetic on quantities that are conceptually complex-number computations underneath.

Looking Ahead: Quantifying “Good” vs. “Bad” Bases

With norms in hand, we can finally give a real, measurable meaning to the “good basis / bad basis” idea flagged repeatedly in this lecture: a good basis has vectors with small norm $\|b_i\|$ that are close to orthogonal; a bad basis for the *same* lattice can have arbitrarily large norms and near-parallel vectors, despite generating an identical set of points. (Here L denotes the **lattice** generated by the basis $\{b_1, \dots, b_n\}$ — all integer combinations of the b_i ’s, i.e. the grid of points they produce; we’ve been using this good-basis/bad-basis idea informally for a few pages now, and Section 8 below is where L finally gets a full, formal definition. For now, just read L as “the lattice this basis describes.”) One common numerical measure of “how good” a basis is is its **orthogonality defect**, $\frac{\prod_i \|b_i\|}{\det(L)}$, where $\det(L)$ is a single positive number — the lattice’s fundamental-cell volume — that depends only on L itself, not on which basis is used to describe it (defined precisely in Section 8.5). A good basis has vectors close in length to $\det(L)^{1/n}$ and close to orthogonal, which drives this ratio down toward its minimum possible value of 1; a bad basis inflates the numerator $\prod_i \|b_i\|$ without changing $\det(L)$ at all, so the ratio grows the more skewed the basis is.

7.3 Gram–Schmidt Orthogonalization

Given *any* basis, can we systematically produce an orthogonal one (spanning the same space, though generally *not* the same lattice — more on that distinction below)? Yes — and the entire idea rests on one simple geometric move: **taking a vector and subtracting off however much of it points along some other direction**, leaving only the part that’s genuinely new. Let’s make that move precise before writing down the full process.

Definition: Vector Projection

The **projection** of a vector v onto a (nonzero) vector u is

$$\text{proj}_u(v) = \frac{\langle v, u \rangle}{\langle u, u \rangle} u.$$

Geometrically, $\text{proj}_u(v)$ is the “shadow” v casts on the line through u — the closest point to v that lies on that line, i.e. the component of v pointing in the u direction. The leftover piece, $v - \text{proj}_u(v)$, is exactly the component of v perpendicular to u : check that $\langle v - \text{proj}_u(v), u \rangle = \langle v, u \rangle - \frac{\langle v, u \rangle}{\langle u, u \rangle} \langle u, u \rangle = 0$, confirming orthogonality directly from the definition.

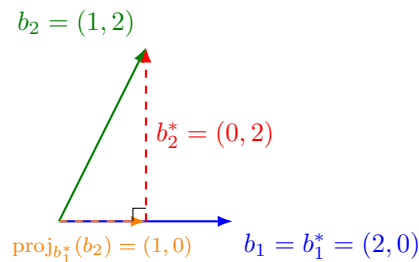


Figure 2: The geometry behind one Gram–Schmidt step (numbers match the worked example below). The green vector b_2 decomposes into an orange piece — its projection onto b_1^* , i.e. “how much of b_2 points along b_1^* ” — and a red perpendicular leftover, $b_2^* = b_2 - \text{proj}_{b_1^*}(b_2)$. That red leftover, translated back to the origin, *is* the new orthogonal basis vector. Every step of Gram–Schmidt is this same picture, repeated once per previous direction.

Definition: Gram–Schmidt Process

Given linearly independent $b_1, \dots, b_k$, define $b_1^* = b_1$, and for $i = 2, \dots, k$:

$$b_i^* = b_i - \sum_{j=1}^{i-1} \mu_{i,j} b_j^*, \quad \text{where } \mu_{i,j} = \frac{\langle b_i, b_j^* \rangle}{\langle b_j^*, b_j^* \rangle}.$$

Each term $\mu_{i,j} b_j^*$ is precisely $\text{proj}_{b_j^*}(b_i)$ from the defbox above — so this formula is nothing more than “**take b_i , and subtract off its projection onto every previously-built orthogonal direction, one at a time.**” Each b_i^* is b_i with the “shadow” it casts on all previous b_j^* subtracted off, leaving only the genuinely new, orthogonal direction. The result $\{b_1^*, \dots, b_k^*\}$ is an orthogonal basis of the same vector space (though, importantly, its integer combinations generally do *not* produce the same lattice as the original b_i ’s — Gram–Schmidt vectors are a vector-space tool, not automatically a lattice basis).

Example: Gram–Schmidt on a 2×2 Example

Let $b_1 = (2, 0)$, $b_2 = (1, 2)$ (the same vectors as Figure 2). Then $b_1^* = b_1 = (2, 0)$. Compute $\mu_{2,1} = \frac{\langle b_2, b_1^* \rangle}{\langle b_1^*, b_1^* \rangle} = \frac{(1)(2) + (2)(0)}{4} = \frac{2}{4} = 0.5$ — this is exactly $\text{proj}_{b_1^*}(b_2) = 0.5(2, 0) = (1, 0)$, the orange vector in the figure. So $b_2^* = b_2 - \text{proj}_{b_1^*}(b_2) = (1, 2) - (1, 0) = (0, 2)$ — the red leftover. Check: $\langle b_1^*, b_2^* \rangle = (2)(0) + (0)(2) = 0$ — orthogonal, as required.

Looking Ahead: Where Gram–Schmidt Leads

This process reappears, in a modified “integer-preserving” form, inside the **LLL algorithm** — the classic algorithm for turning a bad lattice basis into a reasonably good one, and a core tool used both to *attack* weak lattice-based schemes and to reason about the security of well-parametrized ones. We’ll meet it properly in a later lecture; for now, just remember that Gram–Schmidt is the geometric heart of it.

Exercise: Practice

Run Gram–Schmidt on $b_1 = (1, 1)$, $b_2 = (0, 2)$: compute b_1^* , $\mu_{2,1}$, and b_2^* , and verify $\langle b_1^*, b_2^* \rangle = 0$.

8 Lattices: A Formal Definition

Everything so far has allowed *any* scalar from the field F (e.g., any real number) when forming linear combinations. Informally, a lattice is what happens when we impose one dramatic restriction: **only integer combinations are allowed**, which turns a solid, continuous vector space into a discrete grid of isolated points. That informal picture (Figure 4 below) is correct and worth keeping in your head. But the definition we will actually use — the one used in serious treatments of the subject, such as Chris Peikert’s graduate lecture notes *Lattices in Cryptography*¹ — starts from the **subgroup** idea from Section 3.1, not from a basis. We will show the two views agree.

8.1 Discrete Sets

Definition: Discrete Subset

A subset $S \subseteq \mathbb{R}^n$ is **discrete** if every point of S is *isolated*: for every $x \in S$, there exists some $\varepsilon > 0$ such that the open ball $B(x, \varepsilon)$ (all points within distance ε of x) contains *no* point of S other than x itself. Informally: you can always draw a small enough circle around any point of S that catches nothing else from S .

Remark: A Shortcut for Subgroups

Checking every single point of a set for isolation sounds like a lot of work. But if H is a *subgroup* of $(\mathbb{R}^n, +)$ (Section 3.1), there’s a shortcut: it suffices to check isolation *at the origin only* — H is discrete if and only if there is some $\varepsilon > 0$ with $B(\vec{0}, \varepsilon) \cap H = \{\vec{0}\}$. (This is a statement about how much *checking* is needed, not about how many points end up isolated: as the argument below shows, once the origin passes this test, *every* point of H turns out to be isolated too — not just the origin.) *Why this works*: suppose $\vec{0}$ is isolated with radius ε , and suppose some other point $y \in H$ were within ε of some $x \in H$. Since H is a subgroup, $y - x \in H$ as well, and $y - x$ is within ε of $\vec{0}$ — so by our assumption, $y - x = \vec{0}$, i.e., $y = x$. So checking one point (the origin) is enough to certify the whole subgroup is discrete, thanks to closure under subtraction.

¹C. Peikert, *Lattices in Cryptography*, lecture notes, <https://github.com/cpeikert/LatticesInCryptography>.

8.2 The Definition

Definition: Lattice

A **lattice** is a discrete additive subgroup L of $\mathbb{R}^n$. That is, $L \subseteq \mathbb{R}^n$ is a lattice if:

- (L1) L is a subgroup of $(\mathbb{R}^n, +)$ (Section 3.1): it contains $\vec{0}$, and is closed under addition and negation.
- (L2) L is discrete: by the shortcut above, there exists $\varepsilon > 0$ such that $B(\vec{0}, \varepsilon) \cap L = \{\vec{0}\}$.

Example: $\mathbb{Q}$ Is a Subgroup, But *Not* a Lattice

Consider $\mathbb{Q} \subseteq \mathbb{R}$ (the rational numbers) under addition. This *is* a subgroup: it contains 0, the sum of two rationals is rational, and $-a$ is rational whenever a is. So condition (L1) holds. But condition (L2) fails badly: the rationals are **dense** in $\mathbb{R}$, meaning every open ball around 0, no matter how tiny, contains infinitely many other rational numbers. There is no $\varepsilon > 0$ that isolates 0. So $\mathbb{Q}$ is a perfectly good subgroup that is **not** a lattice — this is exactly why condition (L2) has to be part of the definition; subgroup alone is not a strong enough requirement.

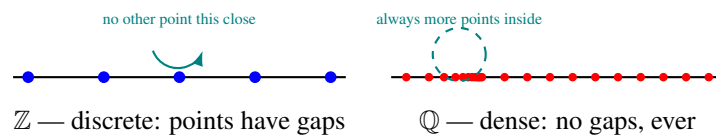


Figure 3: Discreteness, visualized. Left: in $\mathbb{Z}$, every point has a gap of size 1 around it with nothing else nearby — discrete. Right: in $\mathbb{Q}$, no matter how small a circle you draw around a point, there are always infinitely many more rational points crammed inside it — dense, and therefore *not* discrete.

Example: $\mathbb{Z}$ and $\mathbb{Z}^n$ Are Lattices

$\mathbb{Z} \subseteq \mathbb{R}$ is a subgroup (trivially — and recall from Section 3.1 that $n\mathbb{Z}$ is a subgroup of $\mathbb{Z}$ for any n ; here we're just looking at $\mathbb{Z}$ itself sitting inside $\mathbb{R}$), and it is discrete: taking $\varepsilon = 0.5$, $B(0, 0.5) \cap \mathbb{Z} = \{0\}$, since the nearest other integers (± 1) are a full unit away. So $\mathbb{Z}$ is a lattice in $\mathbb{R}$ — the simplest possible example. The same argument, coordinate by coordinate, shows $\mathbb{Z}^n \subseteq \mathbb{R}^n$ (all integer-coordinate points) is a lattice in $\mathbb{R}^n$; it is often called **the integer lattice**.

8.3 The Shortest Vector and the Minimum Distance $\lambda_1(L)$

Before going further, it's worth naming one basic quantity attached to every lattice: how close together its points actually are. We will use this constantly for the rest of the lecture (and the rest of the course), so it earns a formal definition rather than staying an informal aside.

Definition: Shortest Vector and Minimum Distance $\lambda_1(L)$

Let $L \subseteq \mathbb{R}^n$ be a lattice. A **shortest vector** of L is a nonzero vector $v \in L$ whose norm is minimal among all nonzero lattice vectors. The length of a shortest vector is called the lattice's **minimum distance**, or **first successive minimum**, and is defined by

$$\lambda_1(L) = \min_{v \in L \setminus \{\vec{0}\}} \|v\|.$$

The subscript “1” is intentional. More generally, the **successive minima**

$$\lambda_1(L) \leq \lambda_2(L) \leq \dots \leq \lambda_n(L)$$

describe the minimum lengths needed to obtain $1, 2, \dots, n$ linearly independent lattice vectors, respectively. In particular, $\lambda_i(L)$ is the smallest radius r such that the ball

$$B_n(\vec{0}, r) = \{x \in \mathbb{R}^n : \|x\| \leq r\}$$

contains i linearly independent vectors of L .

Thus, $\lambda_1(L)$ is simply the length of a shortest nonzero lattice vector. In this course, we will only need $\lambda_1(L)$ until the optional **Minkowski's Second Theorem** box near the end of this section.

Remark: Why the Minimum Is Actually Attained (Not Just an Infimum)

For a general infinite set, “the smallest nonzero norm” might not literally be achieved by any single element (imagine values getting closer and closer to some limit without ever reaching it). This cannot happen for a lattice, precisely *because* L is discrete (Section 8.1): discreteness guarantees a gap $\varepsilon > 0$ around the origin with no other lattice points inside it, so only *finitely many* lattice points can lie in any bounded region — and the minimum of a norm over a finite set is always achieved. This is also exactly the first step of the “harder direction” proof sketch below (Section 8.4): $\lambda_1(L)$ isn't just a number we can talk about, it's a specific vector we can (in principle) point to.

Remark: Why $\lambda_1(L)$ Matters

Computing $\lambda_1(L)$ exactly — or even just estimating it well — for a lattice given only a (possibly bad) basis is called the **Shortest Vector Problem (SVP)**, and it is believed to be extremely hard once the dimension n is large. This hardness is one of the two or three core hard problems that lattice-based cryptography's security ultimately rests on; we will state SVP formally, and meet its hardness in practice, once we reach the LWE problem in the next lecture.

8.4 Recovering the Basis Picture

The subgroup definition is more general and more standard, but the “grid generated by a basis” picture you may already have in your head is not wrong — it is a theorem, not a definition, and it's worth seeing why.

Fact: Basis Representation of Lattices

Let $L \subseteq \mathbb{R}^n$. Then L is a lattice (in the sense of the subgroup definition above, i.e., a discrete additive subgroup) **if and only if** there exist linearly independent vectors $b_1, \dots, b_k \in \mathbb{R}^n$ (with $k \leq n$) such that

$$L = \mathcal{L}(b_1, \dots, b_k) = \left\{ \sum_{i=1}^k a_i b_i \mid a_i \in \mathbb{Z} \right\}.$$

The vectors $b_1, \dots, b_k$ are called a **basis** of L ; the number k is the **rank** of L . If $k = n$ (the basis spans all of $\mathbb{R}^n$), L is called **full-rank** — this is the common case in cryptography.

Remark: Proof Idea (Sketch Only)

Easy direction ($\Leftarrow$): given linearly independent $b_1, \dots, b_k$, the set of integer combinations is clearly a subgroup (adding two integer combinations, or negating one, gives another integer combination). Discreteness holds because the b_i point in genuinely independent directions: you cannot make $\sum a_i b_i$ arbitrarily close to $\vec{0}$ without every a_i being exactly 0 — linear independence rules out any “near-cancellation.” (Making this fully rigorous uses the fact that the linear map $(a_1, \dots, a_k) \mapsto \sum a_i b_i$ has a bounded inverse on its image.)

Harder direction ($\Rightarrow$): given only that L is a discrete subgroup, we must *construct* a basis. The idea is greedy and inductive: pick b_1 to be a shortest nonzero vector in L — exactly $\lambda_1(L)$ from the defbox above, which exists precisely *because* L is discrete (in a non-discrete set like $\mathbb{Q}$, there is no shortest nonzero element, since you can always find a smaller one). Then look at the part of L not lying on the line through b_1 , and pick b_2 to be a shortest vector there, and so on. A careful argument (using discreteness at each step, and projecting onto the space orthogonal to the vectors already chosen) shows this process terminates with a valid basis after $k \leq n$ steps. The full details are carried out rigorously in Peikert’s notes; we take the result as given here.

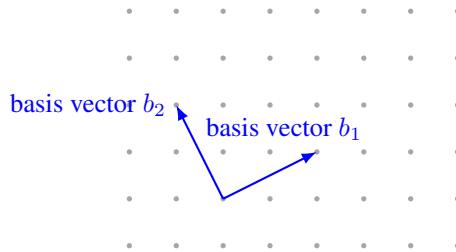


Figure 4: A 2-dimensional, full-rank lattice generated by a basis $\{b_1, b_2\}$: every grid dot is reachable by some *integer* combination $a_1 b_1 + a_2 b_2$ (with $a_1, a_2 \in \mathbb{Z}$), starting from any chosen origin dot. This is the same object as the subgroup definition above — just viewed through its basis instead of as an abstract subgroup.

Remark: Same Lattice, Different Basis

A single lattice has *many* different valid bases (just as a vector space does — recall the “Basis of $\mathbb{R}^2$ ” example in Section 7). Two bases generate the exact same set of points if and only if one can be converted to the other using an integer matrix with determinant ± 1 (a **unimodular** transformation) — we won’t need the details of this fact yet, but the consequence is critical: some bases consist of short, nearly-perpendicular vectors (a “good” basis, easy to compute with), while other, equally valid bases for the *same* lattice consist of long, nearly-parallel vectors (a “bad” basis, hard to compute with). **This gap between good and bad bases for the same lattice is the single most important idea in lattice-based cryptography:** the secret key is typically a good (short) basis, while the public key describes the same lattice using a bad (long, scrambled-looking) basis. Recovering the good basis from the bad one is believed to be extremely hard once the dimension n is large enough.

Example: A Tiny Lattice by Hand

Let $b_1 = (2, 0)$ and $b_2 = (0, 3)$ in $\mathbb{R}^2$. The lattice $\mathcal{L}(b_1, b_2)$ consists of every point $(2a_1, 3a_2)$ for integers a_1, a_2 — so it includes $(0, 0)$, $(2, 0)$, $(-2, 0)$, $(0, 3)$, $(2, 3)$, $(4, -3)$, and so on, but *not* $(1, 1)$ or $(1, 0)$, since those cannot be written as $(2a_1, 3a_2)$ for integer a_1, a_2 . As a sanity check against the subgroup definition directly: is this set a subgroup? Yes — closed under addition and negation, contains $(0, 0)$. Is it discrete? Yes — the nearest point to the origin is at distance 2, so any $\varepsilon < 2$ isolates $\vec{0}$. (Equivalently: $\lambda_1(L) = 2$, achieved by b_1 itself.)

8.5 The Determinant (Volume) of a Lattice

One more piece of structure will matter a great deal in later lectures: every full-rank lattice has a well-defined “volume,” independent of which basis is used to describe it.

Definition: Fundamental Domain and Determinant

Let $L \subseteq \mathbb{R}^n$ be a full-rank lattice with basis $b_1, \dots, b_n$, and let B be the matrix whose columns are $b_1, \dots, b_n$. The **fundamental parallelepiped** associated with this basis is

$$\mathcal{P}(B) = \left\{ \sum_{i=1}^n t_i b_i \mid 0 \leq t_i < 1 \right\}.$$

Here, each coefficient t_i can be *any real number* in the interval $[0, 1)$. Thus, $\mathcal{P}(B)$ consists of all fractional linear combinations of the basis vectors, not merely combinations using coefficients 0 or 1. The **determinant** of the lattice L , denoted by $\det(L)$, is the volume of its fundamental parallelepiped:

$$\det(L) = \text{vol}(\mathcal{P}(B)) = |\det(B)|.$$

The fundamental parallelepiped can be viewed as a basic “tile” for the entire space. Copies of $\mathcal{P}(B)$, shifted by every lattice point $\ell \in L$, tile all of $\mathbb{R}^n$ *exactly*: every point of $\mathbb{R}^n$ lies in *one and only one* translated copy, with no overlap at all. (This is precisely why the definition uses the half-open condition $0 \leq t_i < 1$ rather than $0 \leq t_i \leq 1$: it guarantees each point has a unique representative $p \in \mathcal{P}(B)$, avoiding double-counting along shared faces.) Formally,

$$\mathbb{R}^n = \bigsqcup_{\ell \in L} (\ell + \mathcal{P}(B)),$$

where $\bigsqcup$ denotes a disjoint union. Here, **shifting** (or **translating**) $\mathcal{P}(B)$ by a lattice point ℓ simply means adding ℓ to every point in the parallelepiped:

$$\ell + \mathcal{P}(B) = \{\ell + x \mid x \in \mathcal{P}(B)\}.$$

The shape and volume do not change; only the location changes. **Example: the lattice $\mathbb{Z}^2$.** Consider the lattice

$$L = \mathbb{Z}^2$$

with basis

$$b_1 = (1, 0), \quad b_2 = (0, 1).$$

According to the definition,

$$\mathcal{P}(B) = \{t_1 b_1 + t_2 b_2 \mid 0 \leq t_1, t_2 < 1\}.$$

Substituting the basis vectors gives

$$\begin{aligned} t_1 b_1 + t_2 b_2 &= t_1(1, 0) + t_2(0, 1) \\ &= (t_1, 0) + (0, t_2) \\ &= (t_1, t_2). \end{aligned}$$

Therefore,

$$\mathcal{P}(B) = \{(t_1, t_2) \mid 0 \leq t_1, t_2 < 1\} = [0, 1) \times [0, 1).$$

Notice that t_1 and t_2 are not restricted to the values 0 and 1. They can take any real values between 0 and 1. For example,

$$0.3b_1 + 0.7b_2 = (0.3, 0.7) \in \mathcal{P}(B).$$

Thus, $\mathcal{P}(B)$ contains every point in the unit square $[0, 1) \times [0, 1)$. Now consider the lattice point

$$\ell = (2, 1).$$

Shifting the fundamental parallelepiped by ℓ gives

$$\ell + \mathcal{P}(B) = [2, 3) \times [1, 2).$$

For example,

$$(0, 0) \mapsto (2, 1), \quad (0.3, 0.7) \mapsto (2.3, 1.7).$$

Thus, shifting by $(2, 1)$ simply moves the entire unit square two units to the right and one unit upward. Notice that $\mathcal{P}(B) = [0, 1) \times [0, 1)$ and $\ell + \mathcal{P}(B) = [2, 3) \times [1, 2)$ are entirely different sets of points: they do not share any point in common, illustrating the disjointness of the tiling above. More generally, every point $x \in \mathbb{R}^n$ can be written *uniquely* as

$$x = \ell + p, \quad \ell \in L, \quad p \in \mathcal{P}(B).$$

The lattice point ℓ determines the translated copy of the fundamental parallelepiped, while p determines the position of x within that copy. Since translation does not change volume, every translated copy $\ell + \mathcal{P}(B)$ has the same volume as $\mathcal{P}(B)$. Thus, the determinant of the lattice can be interpreted geometrically as the volume of one fundamental “cell” of the lattice:

$$\boxed{\det(L) = |\det(B)|.}$$

Remark: Why $\det(L)$ Doesn't Depend on Which Basis You Pick

Recall from the “Same Lattice, Different Basis” remark above: any two bases of the same lattice are related by a unimodular matrix U (integer entries, $\det(U) = \pm 1$), i.e., $B' = UB$. Then $\det(B') = \det(U) \det(B) = \pm \det(B)$, so $|\det(B')| = |\det(B)|$. This confirms $\det(L)$ is a genuine property of the lattice itself, not an artifact of which basis happened to be used to compute it — exactly the kind of basis-independence you should expect from something called “the volume of the lattice.”

Example: Computing $\det(L)$ for the Tiny Lattice

For $b_1 = (2, 0)$, $b_2 = (0, 3)$ from the example above, $B = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$, so $\det(L) = |\det(B)| = |2 \times 3 - 0 \times 0| = 6$. Geometrically: the fundamental parallelepiped is just the 2×3 rectangle $[0, 2) \times [0, 3)$, and copies of this rectangle exactly tile the plane, anchored at every lattice point — matching the picture in Figure 4’s style, just for this specific (axis-aligned) lattice.

8.6 Minkowski’s First Theorem: Volume Forces a Short Vector to Exist

We now have the two basic numbers attached to any lattice: $\lambda_1(L)$ (how short its shortest vector is) and $\det(L)$ (how much volume, on average, surrounds each lattice point). Minkowski’s First Theorem is the single most important fact connecting them — and it flips the usual question around. So far, “finding $\lambda_1(L)$ ” has meant: given a specific basis, go and search for a short vector. Minkowski’s theorem asks instead: *without searching at all*, can we predict, from $\det(L)$ alone, an upper bound on how long $\lambda_1(L)$ is *allowed* to be? Surprisingly, yes — and, remarkably, the argument is pure geometry, with no algebra and no knowledge of any particular basis required.

The theorem is about regions $K \subseteq \mathbb{R}^n$ with two specific shape properties, **symmetric** and **convex**, both used informally already in this course but worth pinning down precisely before they appear inside a theorem statement.

Definition: Symmetric Region (Origin-Symmetric / Centrally Symmetric)

A region $K \subseteq \mathbb{R}^n$ is **symmetric** (short for *origin-symmetric*, sometimes called *centrally symmetric* or *point-symmetric*) if

$$K = -K, \quad \text{i.e.,} \quad x \in K \implies -x \in K.$$

In words: whenever a point x belongs to K , its reflection through the origin, $-x$, must belong to K too. Informally, K “looks the same” if you rotate it 180° about the origin — a disk or square *centered at the origin* is symmetric in this sense, but the same disk or square shifted off-center is not (its reflection through the origin lands somewhere else entirely).

Definition: Convex Region

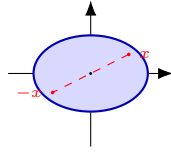
A region $K \subseteq \mathbb{R}^n$ is **convex** if, for every pair of points $x, y \in K$, the entire straight line segment joining them,

$$\{ tx + (1 - t)y : t \in [0, 1] \},$$

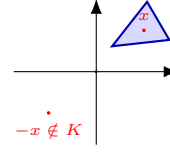
also lies inside K . Informally: a convex region has no “dents,” “notches,” or “holes” — you can never draw a straight line between two points of K that has to leave K and come back. Disks, squares, triangles, and ellipses are all convex; rings (annuli), crescents, and star shapes generally are not.

Remark: The Two Properties Are Independent

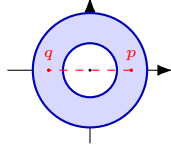
Symmetric and convex describe two *different* things about a region’s shape — one is about “does it look the same reflected through the origin,” the other is about “does it have any dents or holes” — so a region can have either property without the other, both, or neither. Minkowski’s theorem needs *both at once*: Figure 5 below shows one example of each of the four possible combinations.



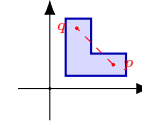
(a) Symmetric *and* convex — e.g. a disk or ellipse centered at the origin.



(b) Convex, but *not* symmetric — $x \in K$ but $-x \notin K$.



(c) Symmetric, but *not* convex — the ring is unchanged by $x \mapsto -x$, but the segment pq cuts through the empty hole.



(d) Neither symmetric nor convex — an off-center, notched (“L-shaped”) region.

Figure 5: The four combinations of symmetric / not, convex / not. In (a), every point’s reflection $-x$ is also in K , and every segment between two points of K stays inside K — this is the shape Minkowski’s theorem needs. In (b), the triangle sits entirely in the upper-right, so a point $x \in K$ has its reflection $-x$ landing in the empty lower-left — symmetry fails, even though the triangle itself has no dents. In (c), the ring *is* symmetric (it looks the same rotated 180°), but it fails convexity: the straight segment between the two marked points p, q passes through the hollow center, which is not part of the ring. In (d), the region is both off-center (so $x \mapsto -x$ maps points outside K) and has an inward notch (so some segments between points of K leave K), failing both properties at once.

Remark: The Idea in One Sentence, Before Any Symbols

If a symmetric, convex region around the origin is made large enough relative to $\det(L)$, it becomes *geometrically impossible* for that region to avoid every nonzero lattice point — so one must be hiding inside it, whether or not you know where.

Definition: Minkowski’s First Theorem

Let $L \subseteq \mathbb{R}^n$ be a full-rank lattice, and let $K \subseteq \mathbb{R}^n$ be a **symmetric** ($K = -K$, i.e. $x \in K \Rightarrow -x \in K$) **convex** region. If

$$\text{Vol}(K) > 2^n \det(L),$$

then K contains a nonzero lattice point, i.e. $K \cap (L \setminus \{\vec{0}\}) \neq \emptyset$.

Remark: Reading the Theorem in Plain Language, Term by Term

- $\det(L)$, as in Section 8.5, is “how much empty space, on average, surrounds each lattice point.”
- $2^n \det(L)$ is a fixed volume threshold, scaled up from $\det(L)$ by a factor that grows with the dimension n (we will see exactly where this factor comes from in the proof below).
- **Symmetric and convex** are conditions on the *shape* of K — a disk or an ellipsoid centered at the origin both qualify; an oddly-shaped or off-center region generally does not.
- **The conclusion** is purely existential: once K ’s volume clears the threshold, K is *guaranteed* to contain some nonzero point of L — but the theorem does not say which point, or how to find it. That “existence without construction” flavor is typical of the deep facts underlying lattice cryptography, and it’s worth getting comfortable with that style of guarantee now.

Remark: A Loose End First: Does K Always Contain the Origin?

Before turning to the proof, it's worth settling a question the theorem's statement doesn't address directly: could a symmetric convex region fail to contain the origin at all? No — and this is worth proving explicitly, because it removes a genuine ambiguity later.

Claim: any nonempty symmetric convex set K contains $\vec{0}$. *Proof:* since K is nonempty, pick any point $x \in K$. By symmetry ($K = -K$), $-x \in K$ too. By convexity, the line segment joining x and $-x$ lies entirely in K — in particular its midpoint, $\frac{1}{2}(x + (-x)) = \vec{0}$, is in K . ■

So $\vec{0} \in K$ is automatic, for *every* symmetric convex K , regardless of its size — it has nothing to do with the volume threshold $2^n \det(L)$ in the theorem. The origin is never at risk of being excluded, so the theorem's volume condition can't be about keeping the origin in; it must be doing something else. That something else is the theorem's real content: guaranteeing a *second, nonzero* lattice point, which is not automatic, and genuinely does depend on K being large enough.

8.6.1 A Fully Formal Proof, via Blichfeldt's Lemma

Remark: Proof Roadmap

The proof splits cleanly into two independent pieces, and it helps to see the two-step strategy before working through either one in detail:

1. **Blichfeldt's Lemma** (a pure volume/pigeonhole argument): *any* region with volume bigger than $\det(L)$ — symmetric, convex, or neither, it doesn't matter — must contain two distinct points whose difference is a nonzero lattice vector. This step never looks at the shape of the region at all.
2. **Bringing back symmetry and convexity:** apply Blichfeldt's Lemma not to K itself, but to the *half-sized* copy $S = \frac{1}{2}K$. The difference of the two points Blichfeldt hands us then turns out, *because* K is symmetric and convex, to land back inside K itself — which is exactly the nonzero lattice point in K that the theorem promises. This is also where the factor of 2^n in the theorem's statement comes from: shrinking K down to $S = \frac{1}{2}K$ divides its volume by 2^n , so S only clears Blichfeldt's threshold ($\text{Vol}(S) > \det(L)$) once $\text{Vol}(K) > 2^n \det(L)$ to begin with.

Fact: Blichfeldt's Lemma

Let $L \subseteq \mathbb{R}^n$ be a full-rank lattice with fundamental domain $\mathcal{P} = \mathcal{P}(L)$ (Section 8.5), so $\text{Vol}(\mathcal{P}) = \det(L)$. Let $S \subseteq \mathbb{R}^n$ be any measurable set with $\text{Vol}(S) > \det(L)$. Then there exist two **distinct** points $p, q \in S$ such that $p - q \in L \setminus \{\vec{0}\}$.

Remark: Proof of Blichfeldt's Lemma

Step 1: Cut S into pieces, one per lattice point, and fold every piece back to the origin. For each lattice point $v \in L$, consider the piece of S that lands in the copy of the fundamental domain sitting at v , i.e. $S \cap (v + \mathcal{P})$, and **fold it back** to the origin's copy by subtracting v :

$$S_v = (S \cap (v + \mathcal{P})) - v \subseteq \mathcal{P}.$$

Step 2: The folded pieces' volumes must add up to more than $\text{Vol}(\mathcal{P})$. Because the translates $\{v + \mathcal{P} : v \in L\}$ tile $\mathbb{R}^n$ with no gaps or overlaps (Section 8.5), every point of S lies in *exactly one* $v + \mathcal{P}$, so the sets $S \cap (v + \mathcal{P})$, as v ranges over L , partition S exactly. Folding back (a rigid shift) doesn't change volume, so

$$\sum_{v \in L} \text{Vol}(S_v) = \sum_{v \in L} \text{Vol}(S \cap (v + \mathcal{P})) = \text{Vol}(S) > \det(L) = \text{Vol}(\mathcal{P}).$$

Step 3: Too much volume packed into one region forces an overlap (pigeonhole). All the S_v 's live inside the single region $\mathcal{P}$. If they were all pairwise disjoint, the sum of their volumes could be at most $\text{Vol}(\mathcal{P})$ — but Step 2 just showed that sum is *strictly greater* than $\text{Vol}(\mathcal{P})$. Contradiction. So some two of them, S_{v_1} and S_{v_2} with $v_1 \neq v_2$, must overlap: there is a point $y \in S_{v_1} \cap S_{v_2}$.

Step 4: Unfold the overlap to recover two points of S whose difference is a lattice vector. By definition of S_v , $y = p - v_1$ for some $p \in S \cap (v_1 + \mathcal{P})$, and $y = q - v_2$ for some $q \in S \cap (v_2 + \mathcal{P})$. Both $p, q \in S$. Since $p - v_1 = q - v_2$, we get

$$p - q = v_1 - v_2 \in L \setminus \{\vec{0}\}$$

(nonzero because $v_1 \neq v_2$, and the difference of two lattice points is again a lattice point, by L 's subgroup closure from Section 3.1). This is exactly the pair the lemma promised. ■

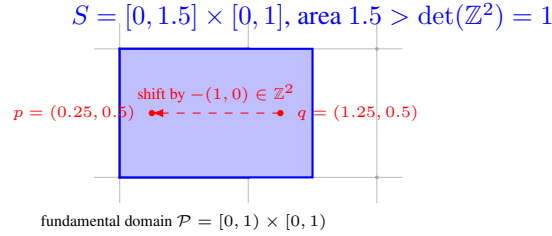


Figure 6: Blichfeldt's Lemma in action for $L = \mathbb{Z}^2$. S (blue) has area 1.5, exceeding $\det(L) = 1$, so it cannot fit inside one fundamental domain without “spilling over.” The spillover — the sliver of S with $x \in [1, 1.5]$ — folds back (shift by $-(1, 0)$) into the same unit square as the rest of S , and lands on top of it: the two marked points $p, q \in S$ fold to the *same* location, and $p - q = (-1, 0)$ is a nonzero lattice vector, exactly as the lemma guarantees.

Example: Blichfeldt's Lemma, Checked by Hand

Take $L = \mathbb{Z}^2$ ($\det(L) = 1$) and $S = [0, 1.5] \times [0, 1]$, area $1.5 > 1$. Following the proof: the piece of S in the fundamental domain $\mathcal{P} = [0, 1) \times [0, 1)$ itself is all of $[0, 1) \times [0, 1)$ (area 1); the piece of S in the neighboring copy $v + \mathcal{P}$ for $v = (1, 0)$ is $[1, 1.5] \times [0, 1)$ (area 0.5), which folds back to $[0, 0.5) \times [0, 1)$ after subtracting v . These two folded pieces — area 1 and area 0.5, total $1.5 > 1 = \text{Vol}(\mathcal{P})$ — cannot both fit disjointly inside $\mathcal{P}$, so they overlap on $[0, 0.5) \times [0, 1)$. Picking any point there, e.g. $(0.25, 0.5)$: this equals $p - (0, 0) = p$ for $p = (0.25, 0.5) \in S$, and also equals $q - (1, 0)$ for $q = (1.25, 0.5) \in S$. Indeed $p, q \in S$ (check: $0.25, 1.25 \in [0, 1.5]$, $0.5 \in [0, 1]$, both confirmed), and $p - q = (-1, 0) \in \mathbb{Z}^2 \setminus \{\vec{0}\}$ — exactly as guaranteed, and matching Figure 6.

Remark: Deriving Minkowski's Theorem from Blichfeldt's Lemma

Now bring back symmetry and convexity to finish the proof, following Step 2 of the roadmap above. Let K be symmetric and convex with $\text{Vol}(K) > 2^n \det(L)$.

Set up the shrunk region. Define $S = \frac{1}{2}K = \{\frac{1}{2}x : x \in K\}$. Volume scales with the n -th power of a linear scale factor, so $\text{Vol}(S) = \text{Vol}(K)/2^n > \det(L)$ — exactly the hypothesis Blichfeldt's Lemma needs.

Apply Blichfeldt's Lemma to S . There exist distinct $p, q \in S$ with $p - q \in L \setminus \{\vec{0}\}$. Since $p, q \in S = \frac{1}{2}K$, write $p = \frac{1}{2}x$ and $q = \frac{1}{2}y$ for (unique) $x, y \in K$. Then

$$p - q = \frac{1}{2}(x - y) = \frac{1}{2}(x + (-y)).$$

Show this difference lands back inside K . Since K is symmetric, $-y \in K$. Since K is convex, the midpoint of x and $-y$ — which is exactly $\frac{1}{2}(x + (-y)) = p - q$ — lies in K . So $p - q$ is simultaneously:

- a nonzero lattice vector (from Blichfeldt's Lemma), and
- a point of K (from symmetry and convexity, just shown).

Therefore $p - q \in K \cap (L \setminus \{\vec{0}\})$ — a nonzero lattice point inside K , exactly as the theorem claims. ■

Remark: Tying the Two Halves Together

Notice how cleanly the labor divides. Blichfeldt's Lemma is the *engine*: pure volume-counting, no geometry of K 's shape involved at all — it would work verbatim for *any* measurable S , symmetric or not, convex or not, as long as it's simply big enough. Symmetry and convexity only enter in the very last step, to upgrade the raw difference $p - q$ (which Blichfeldt hands you for free) into a point that's *also* guaranteed to lie inside K itself, not just somewhere in $\mathbb{R}^n$. This is worth remembering: it's the reason Minkowski's theorem needs *both* hypotheses (symmetric *and* convex) and not just “big enough” — drop either one, and the last step's midpoint trick breaks, even though Blichfeldt's Lemma itself still holds regardless.

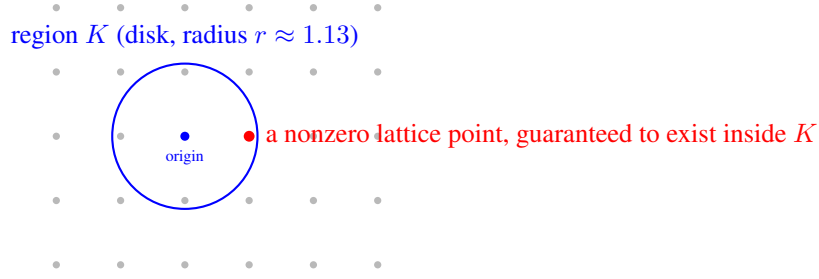


Figure 7: Minkowski’s First Theorem for $L = \mathbb{Z}^2$ ($\det(L) = 1$): once the disk’s area exceeds $2^2 \cdot \det(L) = 4$ (radius $r \gtrsim 1.13$), the theorem guarantees a nonzero lattice point lies inside it — and indeed $(1, 0)$ (shown here relative to the chosen origin) is inside, at distance exactly $1 < 1.13$.

Example: Minkowski’s Bound on the Simplest Lattice, $L = \mathbb{Z}^2$

Take $L = \mathbb{Z}^2$, so $\det(L) = 1$ (Section 8.5). Let K be a disk of radius r centered at the origin — symmetric and convex, exactly as the theorem requires. The theorem applies once $\text{Vol}(K) = \pi r^2 > 2^2 \cdot \det(L) = 4$, i.e. once

$$r > \frac{2}{\sqrt{\pi}} \approx 1.128.$$

So Minkowski’s theorem *guarantees*, without any further work, that a disk of radius 1.13 around the origin must contain a nonzero lattice point. Checking directly: we already know $\lambda_1(\mathbb{Z}^2) = 1$ (the point $(1, 0)$), and indeed $1 < 1.128$ — consistent with, and in fact comfortably inside, the guarantee. Notice the theorem didn’t need to know that $(1, 0)$ was the answer; it only needed the disk to be big enough.

Example: The Same Bound on a Very Different-Looking Lattice

Now take a stretched lattice with basis $b_1 = (5, 0)$, $b_2 = (0, 1)$, so $\det(L) = |5 \times 1 - 0 \times 0| = 5$ (Section 8.5). The disk threshold becomes $\pi r^2 > 4 \times 5 = 20$, i.e. $r > \sqrt{20/\pi} \approx 2.523$. Minkowski’s theorem guarantees a nonzero lattice point within distance 2.523 of the origin. The actual shortest vector here is $(0, 1)$, of length 1 — again comfortably inside the guaranteed radius, but this time the guarantee is quite *loose*: the theorem promised “within 2.523” and reality delivered “within 1.” This is expected and normal — Minkowski’s theorem is a **worst-case guarantee**, not an exact formula; it never overpromises (the bound always holds), but it can certainly underdeliver on precision for any one specific lattice. Its real value is as a *universal* ceiling that holds for *every* lattice of a given determinant, not as a precise estimate for one.

Remark: From the Disk Bound to the General Formula

Redo the disk calculation above in n dimensions instead of 2: the volume of a radius- r ball in $\mathbb{R}^n$ shrinks (relative to r^n) as n grows, and pushing the same threshold argument through (we omit the n -dimensional volume formula itself, which involves the Gamma function) yields the general bound already stated in Lecture 1's introduction to this topic:

$$\lambda_1(L) \leq \sqrt[n]{n} \cdot \det(L)^{1/n} \quad (\text{up to a small constant factor}).$$

In plain terms: a lattice can never “hide” an unexpectedly long shortest vector behind a small determinant — volume and minimum distance are locked together by geometry alone, regardless of which basis you're handed to describe the lattice. This is part of *why* SVP is only believed hard in the specific regime cryptographers deliberately choose (large dimension n , carefully scaled $\det(L)$), rather than for arbitrary lattices — a preview of the parameter-selection discussion we'll return to once we look at how Kyber and Dilithium choose their concrete dimensions and moduli.

Exercise: Practice

Let $L = \mathbb{Z}^3$ (so $\det(L) = 1$). Using the volume of a ball in $\mathbb{R}^3$, $\text{Vol} = \frac{4}{3}\pi r^3$, find the radius r at which Minkowski's theorem first guarantees a nonzero lattice point inside the ball. Compare it to the true value $\lambda_1(\mathbb{Z}^3) = 1$ (e.g. the point $(1, 0, 0)$) — is the guarantee looser or tighter, relative to the truth, than it was for $\mathbb{Z}^2$ in the worked example above?

Looking Ahead: Optional / Advanced: Minkowski's Second Theorem

The First Theorem only talks about $\lambda_1(L)$, the *single* shortest vector. It says nothing about the second-shortest independent direction, the third, and so on. **Minkowski's Second Theorem** fills that gap: recall the **successive minima** $\lambda_1(L) \leq \lambda_2(L) \leq \dots \leq \lambda_n(L)$ flagged in the defbox above, where $\lambda_i(L)$ is the smallest radius r such that the ball of radius r contains i *linearly independent* lattice vectors. The theorem bounds their *product*:

$$\frac{2^n}{n!} \det(L) \leq \lambda_1(L) \cdots \lambda_n(L) \leq 2^n \det(L).$$

This is not core material — everything in Sessions 1–6 (understanding LWE, the algorithms, and the physical attacks in Section 2 of the proposal) only ever leans on the First Theorem's intuition: “small determinant $\Rightarrow$ short vector must exist.” The Second Theorem becomes genuinely useful only once the research moves toward analyzing *trapdoor* lattices and Gaussian sampling in depth — which is exactly what underlies Falcon's signing procedure (flagged for Session 5). A trapdoor basis needs *all* of its vectors short, not just the shortest one, for Gaussian sampling to be both efficient and secure; that “all vectors short together” requirement is precisely what the successive minima $\lambda_1, \dots, \lambda_n$ capture, and the Second Theorem is the tool that ties them back to $\det(L)$. This box is worth returning to *if and only if* the Phase 2 research direction (Section 4 of the proposal) ends up involving Falcon's sampler or trapdoor generation in detail — otherwise, the First Theorem alone is sufficient background.

Exercise: Practice

For the lattice generated by $b_1 = (1, 2)$, $b_2 = (3, 1)$ (from the earlier homework exercise): compute $\det(L)$ directly from the basis matrix. Then pick any *other* valid basis for the same lattice (e.g., $b'_1 = b_1$, $b'_2 = b_2 - b_1$) and confirm you get the same $\det(L)$.

Looking Ahead: Where This Leads Next Lecture

With Groups $\rightarrow$ Subgroups $\rightarrow$ Rings $\rightarrow$ Fields $\rightarrow$ Modular arithmetic $\rightarrow$ Vector spaces $\rightarrow$ Basis $\rightarrow$ Lattice now in place, the next lecture can finally state the **Learning With Errors (LWE)** problem precisely: a secret vector, hidden inside noisy linear equations over $\mathbb{Z}_q$ (a ring, Section 6), which is exactly the kind of “bad basis” hardness described above, dressed up in a specific, standardized cryptographic form. We will also unpack the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ flagged in Section 4, since **module lattices** (lattices whose coordinates come from a ring like R_q instead of plain integers) are exactly what Kyber and Dilithium use in practice.

9 Summary Table

Structure	Requires	Example
Group	1 operation, inverses exist	$(\mathbb{Z}, +)$
Subgroup	A subset that is itself a group	$2\mathbb{Z} \subseteq \mathbb{Z}$
Coset / Quotient group	G sorted into shifted copies of H	$\mathbb{Z}/3\mathbb{Z} = \{0, 1, 2\}$
Ring	2 operations, no division needed	$(\mathbb{Z}, +, \times), \mathbb{Z}[x]$
Integral domain	Ring, no zero divisors	$\mathbb{Z}, \mathbb{Z}_5$ (not $\mathbb{Z}_6$)
Field	Ring + division by nonzero elements	$\mathbb{Q}, \mathbb{R}, \mathbb{Z}_p$ (p prime)
Modulus	Wraparound arithmetic	$\mathbb{Z}_n = \{0, \dots, n-1\}$
Vector space	Field + vectors + scalar mult.	$\mathbb{R}^n$
Basis	Independent, spanning vector set	$\{(1, 0), (0, 1)\}$ in $\mathbb{R}^2$
Orthogonal basis	Basis, all pairs perpendicular	Gram–Schmidt output
Lattice	Discrete additive subgroup of $\mathbb{R}^n$	$\mathbb{Z}^n$; grid of points
$\lambda_1(L)$	Length of the shortest nonzero lattice vector	$\lambda_1(\mathbb{Z}^2) = 1$
$\det(L)$	Volume of the fundamental domain	$ \det(B) $, any basis B

10 Full List of Exercises (Assign as Homework Before Lecture 2)

Exercise: Homework Set

1. Verify $(\{1, 2, 3, 4\}, \times \bmod 5)$ is a group by writing out its full multiplication table (Section 3 exercise, above).
2. Is $3\mathbb{Z} = \{\dots, -6, -3, 0, 3, 6, \dots\}$ a subgroup of $(\mathbb{Z}, +)$? Justify using the subgroup test (Section 3.1). Is $\{0, 3, 6, 9, \dots\}$ (only the non-negative multiples of 3) a subgroup of $(\mathbb{Z}, +)$? Why or why not?
3. List all distinct cosets of $4\mathbb{Z}$ in $\mathbb{Z}$, confirm there are exactly 4, and identify which coset -5 belongs to (Section 3.2 exercise, above).
4. Find a pair of zero divisors in $\mathbb{Z}_{12}$, and use the zero-divisor fact (Section 4.1) to explain why $\mathbb{Z}_{12}$ cannot be a field, without computing any inverses.
5. Compute $23 \times 15 \bmod 7$; list the non-invertible elements of $\mathbb{Z}_8$; use the Extended Euclidean Algorithm (Section 6.1) to find the inverse of $3 \bmod 7$ — confirm it matches the worked example.
6. Use the Extended Euclidean Algorithm to find $5^{-1} \bmod 11$ (Section 6.1 exercise, above), showing all steps.
7. Is $\mathbb{Z}_9$ a field? Why or why not? Answer it two ways: (a) using gcd (Section 6), and (b) using zero divisors (Section 4.1).
8. Show that $\{(1, 1), (1, -1)\}$ is a valid basis of $\mathbb{R}^2$ by (a) checking linear independence and (b) writing the vector $(4, 2)$ as a linear combination of them.
9. Run Gram–Schmidt on $b_1 = (1, 1)$, $b_2 = (0, 2)$ (Section 7.3 exercise, above), and verify the result is orthogonal.
10. For the lattice generated by $b_1 = (1, 2)$ and $b_2 = (3, 1)$: (a) is the point $(7, 5)$ in the lattice? (b) compute $\det(L)$ from this basis; (c) pick a different valid basis for the same lattice and confirm $\det(L)$ is unchanged (Section 8.5 exercise, above).
11. Let $L = \{(x, y) \in \mathbb{Z}^2 : x + y \text{ is even}\}$. (a) Show L is a subgroup of $(\mathbb{R}^2, +)$. (b) Show L is discrete (find an ε that isolates $\vec{0}$). (c) Since L is therefore a lattice, find a basis $\{b_1, b_2\}$ for it, and compute $\det(L)$.
12. In your own words (3–4 sentences): why does the definition of a lattice need *both* “subgroup” and “discrete”? Use the $\mathbb{Q}$ example from Section 8.2 to explain what goes wrong if you drop the discreteness requirement.
13. In your own words (3–4 sentences), explain why a “good basis” and a “bad basis” for the same lattice can make the same mathematical object feel completely different to work with computationally, now that you can measure this with norms (Section 7.2).